



Nautilus Playground

A cohesive environment for prototyping, testing, and demonstrating trading strategies

Press Space for next page →



What is Nautilus Trader?

- Open-source high-performance algorithmic trading platform
- Production-grade system for backtesting and live trading
- AI-first design for developing and deploying trading strategies
- Core written in Rust with Python bindings for performance and safety
- Universal and asset-class-agnostic supporting multiple venues simultaneously

Key Features

- Fast: Core in Rust with async networking
- Reliable: Type and thread safety
- Portable: OS independent (Linux, macOS, Windows)
- Flexible: Modular adapter architecture
- Advanced: Sophisticated order types and execution

Architecture

- Event-driven design
- Message bus for component communication
- Domain-driven design principles
- Ports and adapters pattern
- Crash-only design philosophy

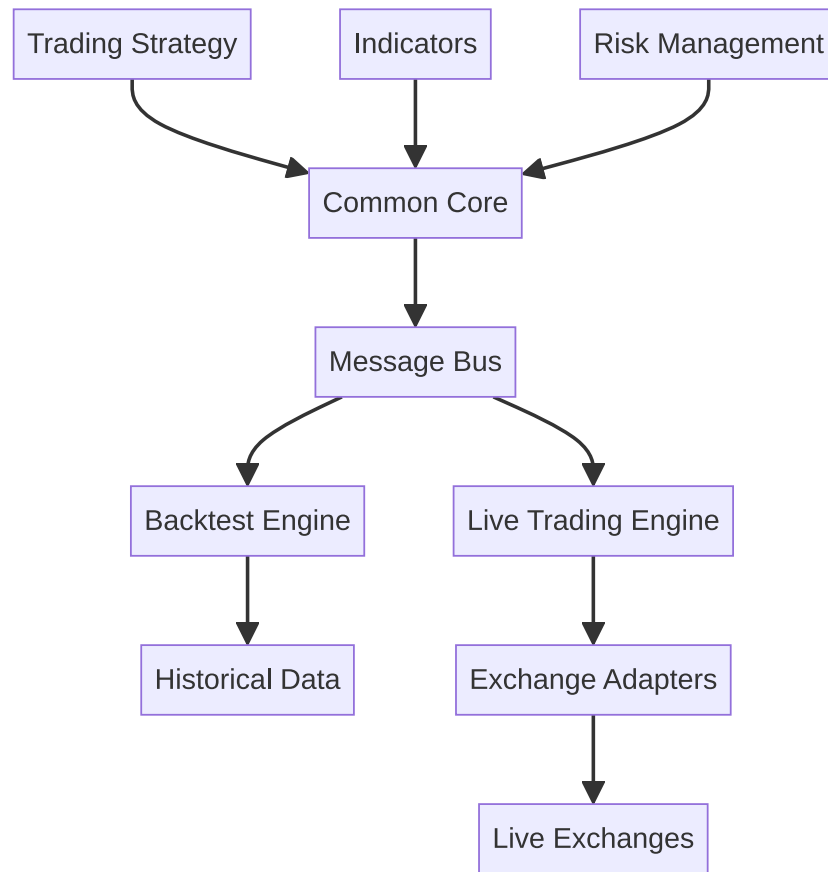
Nautilus Trader Architecture

Environment Contexts

- ➡ **Backtest**: Historical data with simulated venues
- ➡ **Sandbox**: Real-time data with simulated venues
- ➡ **Live**: Real-time data with live venues

Common Core

- ➡ Shared code between all environments
- ➡ Modular components with clean interfaces
- ➡ Efficient `MessageBus` for communication
- ➡ Single-thread design for optimal performance



Nautilus Trader Technical Details

Data Types

- ➡ `OrderBookDelta` (L1/L2/L3)
- ➡ `OrderBookDepth10`
- ➡ `QuoteTick`
- ➡ `TradeTick`
- ➡ `Bar`
- ➡ `Instrument`
- ➡ `InstrumentStatus`

Bar Aggregations

- ➡ Time-based: `MILLISECOND` to `MONTH`
- ➡ Volume-based: `TICK`, `VOLUME`, `VALUE`
- ➡ Advanced: `IMBALANCE`, `RUNS`

Order Types

- ➡ `MARKET`
- ➡ `LIMIT`
- ➡ `STOP_MARKET`
- ➡ `STOP_LIMIT`
- ➡ `MARKET_TO_LIMIT`
- ➡ `MARKET_IF_TOUCHED`
- ➡ `LIMIT_IF_TOUCHED`
- ➡ `TRAILING_STOP_MARKET`
- ➡ `TRAILING_STOP_LIMIT`

Account Types

- ➡ `Cash` (single/multi-currency)
- ➡ `Margin` (single/multi-currency)

Nautilus Trader Strategy Example

Strategy Implementation

- Strategies inherit from `Strategy` class
- Configuration via `StrategyConfig`
- Event-driven architecture
- Handlers for market data and events
- Built-in position management
- Comprehensive indicator framework

```
class MACDStrategy(Strategy):
    def __init__(self, config: MACDConfig):
        super().__init__(config=config)
        # Initialize MACD indicator
        self.macd = MovingAverageConvergenceDivergence(
            fast_period=config.fast_period,
            slow_period=config.slow_period,
            price_type=PriceType.MID
        )
        self.position = None

    def on_start(self):
        # Subscribe to market data
        self.subscribe_quote_ticks(self.config.instrument_)

    def on_quote_tick(self, tick: QuoteTick):
        # Update indicator with new data
        self.macd.handle_quote_tick(tick)

        if not self.macd.initialized:
            return # Wait for indicator warmup

        # Trading logic
        if self.macd.value > 0.0001: # Buy signal
            self.buy()
        elif self.macd.value < -0.0001: # Sell signal
            self.sell()
```

What is Nautilus Playground?

A self-contained environment for algorithmic trading development using the Nautilus Trader framework

- Develop and test trading strategies
- Backtest strategies with historical data
- Run live trading simulations
- Explore and visualize trading results
- Learn algorithmic trading concepts

Key Features

- □ Comprehensive backtesting framework
- □ Strategy development environment
- □ Live trading capabilities
- □ Data visualization tools
- □ Docker-based development environment

Tooling & Libraries for Quants and Data Scientists

Data Analysis & ML Libraries

- pandas, numpy, scipy - Core data analysis
- pandas-ta - 130+ technical indicators
- statsmodels - Statistical models, tests, and analysis
- scikit-learn, xgboost, tensorflow - Machine learning
- pytorch, ray - Deep learning and distributed computing
- prophet, darts - Time series forecasting
- QuantStats - Portfolio analytics and performance metrics

Visualization Tools

- matplotlib, seaborn - Static visualizations

Developer Productivity

- VS Code with Python, Jupyter, and GitHub Copilot extensions
- JupyterLab - Interactive computing environment
- Neovim with NvChad - Advanced text editor
- Docker - Containerized development environment
- Git - Version control
- pytest - Testing framework
- black, isort - Code formatting
- pylint - Code linting

Specialized Trading Tools

- Ornstein-Uhlenbeck process modeling - For mean reversion

Project Structure

- ➔ `.devcontainer/` : Development container configuration
- ➔ `data/` : Data files for backtesting
 - ➔ `catalog/` : Local data catalog
- ➔ `docs/` : Project documentation
- ➔ `notebooks/` : Jupyter notebooks
- ➔ `scripts/` : Utility scripts
- ➔ `src/` : Source code
 - ➔ `execution/` : Execution algorithms
 - ➔ `indicators/` : Technical indicators
 - ➔ `strategies/` : Trading strategies
- ➔ `tests/` : Unit and integration tests

Development Environment

- ➔ Based on Docker container
- ➔ VS Code integration
- ➔ Pre-installed libraries:
 - ➔ `nautilus_trader`
 - ➔ `pandas`, `numpy`, `scipy`
 - ➔ `matplotlib`, `plotly`, `seaborn`
 - ➔ `scikit-learn`, `xgboost`, `tensorflow`
 - ➔ And many more...

Getting Started

```
# Clone the repository
git clone https://github.com/your-repo/nautilus-playground

# Open in VS Code and reopen in container
```

Moving Average Crossover Strategy

- ➡ Simple yet effective strategy
- ➡ Based on two Exponential Moving Averages (EMAs)
- ➡ Trading signals generated at crossovers
- ➡ Implemented in

src/strategies/moving_average_crossover/

```
from nautilus_trader.model.enums import OrderSide
from nautilus_trader.trading.strategy import Strategy

class MovingAverageCrossover(Strategy):
    def __init__(self, config):
        self.fast_ema = PandasTaIndicator(
            bar_type=self.bar_type,
            indicator_name="ema",
            params={"length": self.fast_ema_period},
        )

        self.slow_ema = PandasTaIndicator(
            bar_type=self.bar_type,
            indicator_name="ema",
            params={"length": self.slow_ema_period},
        )
```

Signal Logic

- ➡ Buy Signal: Fast EMA crosses above Slow EMA
- ➡ Sell Signal: Fast EMA crosses below Slow EMA
- ➡ Position management with market orders
- ➡ Configurable trade size and EMA periods

```
# Check for bullish crossover (fast crosses above slow)
if (self.previous_fast_ema <= self.previous_slow_ema and
    current_fast_ema > current_slow_ema):
    self._log.info("Bullish crossover detected")
    self._handle_signal(OrderSide.BUY)

# Check for bearish crossover (fast crosses below slow)
```

Trading Strategies

The project includes two main trading strategies:

Moving Average Crossover

A simple yet effective strategy based on EMA crossovers.

Features:

- Generates buy signals when fast EMA crosses above slow EMA
- Generates sell signals when fast EMA crosses below slow EMA
- Configurable parameters: fast/slow periods, trade size

Mean Reversion with ADX Filter

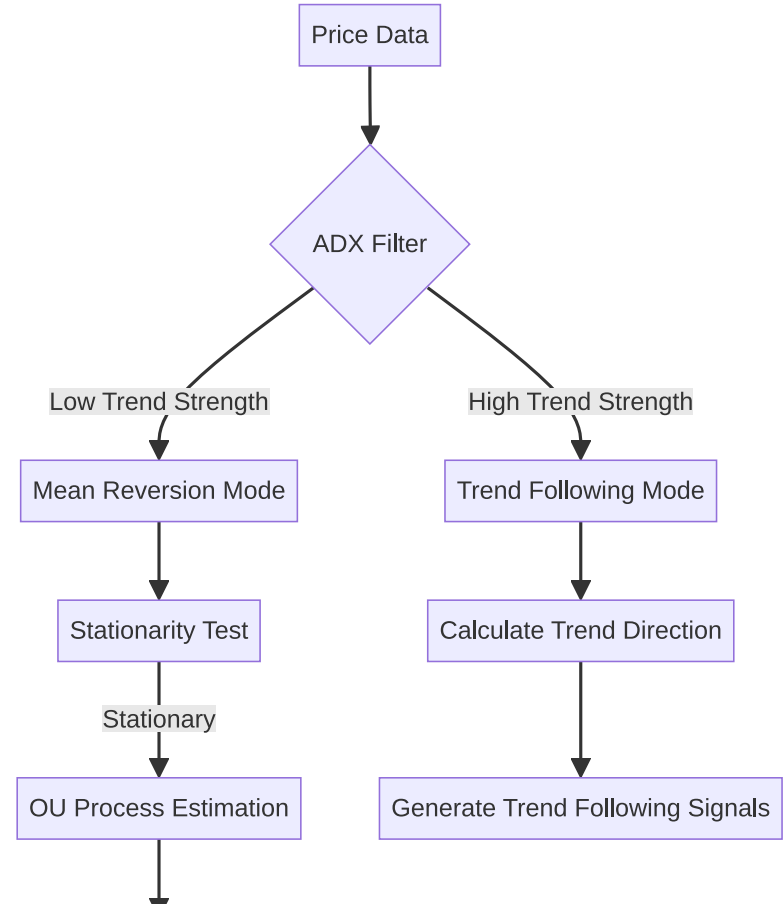
A sophisticated strategy combining statistical methods and machine learning.

Features:

- Uses ADX to determine trend strength
- Applies mean reversion in range-bound markets
- Applies trend following in trending markets
- Uses Ornstein-Uhlenbeck process to model price movements
- Includes machine learning enhancements

Mean Reversion Strategy

- Sophisticated strategy combining multiple approaches
- Uses ADX (Average Directional Index) as trend filter
- Applies statistical tests for mean reversion
- Implements Ornstein-Uhlenbeck process modeling
- Includes machine learning for signal enhancement



Technical Indicators

PandasTaIndicator

- Wrapper for the pandas-ta library
- Provides access to 130+ technical indicators
- Seamlessly integrates with Nautilus Trader
- Located in `src/indicators/pandas_ta_indicator.py`

Supported Indicator Types

- Trend Indicators: ADX, MACD, Moving Averages
- Momentum Indicators: RSI, Stochastic, CCI
- Volatility Indicators: Bollinger Bands, ATR
- Volume Indicators: OBV, MFI
- And many more...

```
# Create an RSI indicator
rsi = PandasTaIndicator(
    bar_type=bar_type,
    indicator_name="rsi",
    params={"length": 14},
    price_type=PriceType.LAST,
)

# Create an ADX indicator
adx = PandasTaIndicator(
    bar_type=bar_type,
    indicator_name="adx",
    params={"length": 14},
    price_type=PriceType.LAST,
)
```

Execution Algorithms

TWAP (Time-Weighted Average Price)

- Executes orders over a specified time period
- Aims to achieve a price close to the time-weighted average
- Minimizes market impact for large orders
- Located in `src/execution/twap.py`

How TWAP Works

1. Divides the order into smaller chunks
2. Executes these chunks at regular intervals
3. Reduces market impact and slippage
4. Useful for large orders in less liquid markets

```
class TWAP(ExecutionAlgorithm):  
    def __init__(  
        self,  
        instrument_id: InstrumentId,  
        order_side: OrderSide,  
        quantity: Quantity,  
        start_time: pd.Timestamp,  
        end_time: pd.Timestamp,  
        interval_minutes: int = 5,  
    ):  
        # Initialize TWAP parameters  
        self.start_time = start_time  
        self.end_time = end_time  
        self.interval_minutes = interval_minutes  
        self.intervals = self._calculate_intervals()
```

Jupyter Notebooks

The project includes Jupyter notebooks for exploring and demonstrating strategies:

1. Moving Average Crossover Backtest

- Located in `notebooks/moving_average_crossover_backtest.ipynb`
- Demonstrates backtesting and optimizing the strategy
- Includes performance analysis and visualization

2. Mean Reversion Strategy Exploration

- Located in `notebooks/mean_reversion_strategy_exploration.ipynb`
- Explores mean reversion characteristics in price data
- Tests for stationarity using Augmented Dickey-Fuller test
- Estimates Ornstein-Uhlenbeck process parameters
- Visualizes mean reversion opportunities
- Analyzes performance metrics

Backtesting

Run backtests using the `main_backtest.py` script:

```
# First download the required data
python scripts/download_data.py --symbols BTCUSDT --timeframes 1h 1d --days 90

# Then run the backtest
python src/main_backtest.py --config path/to/config.yaml
```

Specify date ranges:

```
python src/main_backtest.py --config path/to/config.yaml --start-date 2023-01-01 --end-date 2023-12-31
```

Backtest Configuration

- Configure strategies using YAML files
- Set parameters like trade size, indicator periods, etc.
- Specify instruments and timeframes
- Define risk management parameters

Live Trading

Run live trading using the `main_live.py` script:

```
# For live trading
python src/main_live.py

# For paper trading (using Binance testnet)
python src/main_live.py --paper

# With custom configuration
python src/main_live.py --config path/to/config.yaml
```

Setup Requirements

1. Create a `.env` file with your API credentials
2. Configure the strategy parameters
3. Start the trading node
4. Monitor performance in real-time

Supported Exchanges

- ➡ Binance (Spot, Margin, Futures)
- ➡ More exchanges can be added through Nautilus Trader adapters



Thank You!

GitHub Repository · Documentation

Press Space for next page →

